

# 4\_parameter\_serwer\_cwiczenia-Copy1

March 28, 2022

## 0.1 Terminal - polecenia

Zadania należy wykonać dla poniższych parametrów

Nazwa parametru    Wartość    /description "opis symulacji"    /goal/tolerance/x 0.03    /goal/tolerance/y 0.01    /robots/robot1/tryb "auto"    /robots/robot2/tryb "manual"    /robots/robot3/tryb "auto"    /robots/robot3/id 1    /robots/robot3/isActive True

1. Wykorzystać polecenie `rosparam set` do ustawiania wartości

```
[3]: !rosparam set /description: "opis symulacji"
```

```
[5]: !rosparam set /goal/tolerance/y 0.01
```

```
[6]: !rosparam set /robots/robot3/isActive False
```

2. Ustawić parametry z wykorzystaniem słownika (1 polecenie) dla `/robots/robot3`

```
[9]: !rosparam set /robots/robot3 "{id: 2,isActive: False, tryb: \"manual\"}"
```

```
[15]: !rosparam set /robots/robot3 "{id: 2, isActive: False, tryb: 'manual'}"
```

```
[38]: !rosparam list
```

```
/gains/d
/gains/i
/gains/p
/kolor_tla/background_b
/kolor_tla/background_g
/kolor_tla/background_r
/robot_2/omega_max
/robot_2/v_max
/robot_3/goal_tolerance
/robot_3/omega_max
/robot_3/tryb
/robot_3/v_max
/robot_4/goal_tolerance/angle
/robot_4/goal_tolerance/x
/robot_4/goal_tolerance/y
```

```

/robot_4/omega_max
/robot_4/tryb
/robot_4/v_max
/robot_on
/robots/robot1/tryb
/robots/robot2/tryb
/robots/robot3/id
/robots/robot3/isActive
/robots/robot3/tryb
/rosdistro
/roslaunch/uris/host_localhost__43007
/rosversion
/run_id
/turtlesim/background_b
/turtlesim/background_g
/turtlesim/background_r

```

3. Utworzyć plik .yaml i zaimportować dane

```
[17]: !rosparam load param_cw3.yaml
```

4. Zapisać wszystkie parametry do pliku.

```
[20]: !rosparam dump params2137.yaml
```

5. Zapisać parametry od robotów do pliku robots.yaml

```
[22]: !rosparam dump -v nazwa_pliku.yaml /robots
```

```

dumping namespace [/robots] to file [nazwa_pliku.yaml]
/robots/robot3/isActive=True
/robots/robot3/id=1
/robots/robot3/tryb=auto
/robots/robot1/tryb=auto
/robots/robot2/tryb>manual

```

6. Zapisać parametry z tolerancją osiągnięcia celu do pliku goal.yaml

```
[26]: !rosparam dump -v nazwa_pliku2.yaml /robot_4/goal_tolerance
```

```

dumping namespace [/robot_4/goal_tolerance] to file [nazwa_pliku2.yaml]
/robot_4/goal_tolerance/x=0.1
/robot_4/goal_tolerance/y=0.05
/robot_4/goal_tolerance/angle=3.141592653589793

```

```
[39]: !rosparam list
```

```

/rosdistro
/roslaunch/uris/host_localhost__36377

```

```
/rosversion
/run_id
```

7. Usunąć parametr description

```
[31]: !rosparm delete -v /description
```

```
deleted parameter [/description]
```

8. Usunąć przestrzeń nazw od goal

```
[32]: !rosparm delete -v /goal
```

```
deleted parameter [/goal]
```

## 0.2 Obsługa w języku Python

```
[40]: import rospy
```

Zadania należy wykonać dla poniższych parametrów

Nazwa parametru    Wartość    /description "opis symulacji"    /goal/tolerance/x 0.03 /goal/tolerance/y 0.01 /robots/robot1/tryb "auto" /robots/robot2/tryb "manual" /robots/robot3/tryb "auto" /robots/robot3/id 1 /robots/robot3/isActive True

1. Ustawić wartości parametrów z wykorzystaniem rospy.set\_param dla dla robots

```
[42]: rospy.set_param("description", "opis symulacji")
rospy.set_param("robots", {"robot3": {"tryb": "auto", "id": 1, "isActive":
↪True}})
```

2. Ustawić wartości parametru /goal z wykorzystaniem rosparm.set\_param dla dla goal

```
[48]: import rosparm
rosparm.set_param("goal", "{\"tolerance\": {'x': 0.03, 'y': 0.01}}")
```

3. Ustawić wartość parametru description

```
[49]: rosparm.set_param("description", "opis symulacji")
```

4. sprawdzić czy parametr /robots/robot1/tryb istnieje?

```
[50]: rospy.has_param("/robots/robot1/tryb")
```

```
[50]: False
```

5. sprawdzić czy parametr /robots/robot1/isActive istnieje?

```
[51]: rospy.has_param("/robots/robot3/isActive")
```

[51]: True

6. Wyszukać po nazwie parametry dla robot3.

```
[52]: param = rospy.search_param("robot3")
      v = rospy.get_param(param)
```

```
[53]: param
```

```
[54]: v
```

```
[54]: {'run_id': 'e7b7d13e-aa80-11ec-9f3b-0242ac130002',
      'roslaunch': {'uris': {'host_localhost__36377': 'http://localhost:36377/'}},
      'rosversion': '1.15.13\n',
      'rostdistro': 'noetic\n',
      'description': 'opis symulacji',
      'robots': {'robot3': {'tryb': 'auto', 'id': 1, 'isActive': True}},
      'goal': {'tolerance': {'x': 0.03, 'y': 0.01}}}
```

7. Usunąć parametry dla robot1.

```
[57]: rospy.delete_param("/goal")
```

8. Usunąć jednym poleceniem parametry dla robot3.

```
[58]: rospy.delete_param("/robots/robot3/tryb")
```

9. Usunąć jeszcze raz parametry dla robot1 i wyświetlić komunikat, że parametr nie istnieje.

```
[60]: try:
      rospy.delete_param("/robots/robot3/tryb")
except KeyError:
    print("Nie ma parametru")
```

Nie ma parametru

### 0.3 Zadanie dodatkowe

1. Skopiować klasę Robot z zajęć o serwisach wraz z importem odpowiednich bibliotek. Rozbudować ją o napisaną w poprzednim zadaniu funkcję. Podczas inicjalizacji klasy Robot powinny się również pojawić parametry robota. Funkcja jako metoda klasy powinna mieć jako pierwszy argument self wskazujący na klasę.
2. Dopisać metodę do wyświetlania parametrów robota z jego przestrzeni nazw.
3. Uzupełnić metodę **unregistered** o usuwanie wszystkich parametrów robota.
4. Dodać do klasy serwis od usuwania robota. (Serwis /kill)

5. Utworzyć metodę do wyświetlania listy parametrów robota. Odczytać parametry z przestrzeni nazw i wyświetlić.

[ ]: